

Relazione sul Progetto di Edge Detection

1. Introduzione

L'**edge detection** rappresenta una delle operazioni fondamentali nell'ambito dell'image processing, poiché consente di individuare le discontinuità di intensità all'interno di un'immagine, corrispondenti spesso ai contorni degli oggetti presenti nella scena. L'estrazione accurata dei bordi costituisce un passaggio chiave per numerose applicazioni, tra cui segmentazione, riconoscimento di oggetti e analisi delle immagini.

Questo progetto consiste nello sviluppo di un programma in Python che permette di sperimentare e confrontare differenti tecniche di **edge detection**, sia classiche che basate su **deep learning**, all'interno di un ambiente interattivo e modulare. L'obiettivo principale è quello di analizzare il comportamento dei diversi algoritmi.

In particolare, il progetto include l'implementazione e l'utilizzo dei seguenti metodi:

- * **Canny**, **algoritmo** classico di **edge detection**, implementato interamente da zero, senza l'uso di librerie esterne dedicate, seguendo fedelmente la formulazione teorica e i passaggi standard dell'**algoritmo**;

- * **TEED** (Tiny and Efficient Edge Detector), un modello di **edge detection** basato su reti neurali convoluzionali, progettato per essere efficiente e robusto;

- * **DexiNed** (Dense Extreme Inception Network for **Edge Detection**), un modello **deep learning** più complesso, in grado di catturare bordi a differenti scale e livelli semantici.

Il software sviluppato consente di caricare immagini singole o **dataset**, selezionare dinamicamente l'**algoritmo** da utilizzare e visualizzare in modo immediato i risultati dell'**edge detection**, permettendo così un confronto diretto tra approcci tradizionali e moderni.

Attraverso questo progetto si intende fornire uno strumento utile sia a scopo didattico sia sperimentale, evidenziando l'evoluzione delle tecniche di **edge detection** dal punto di vista algoritmico e applicativo.

2. Dataset utilizzati: BIPED e BSDS500

Nel presente progetto sono stati utilizzati due **dataset** di riferimento per il problema dell'**edge detection**: BIPED e BSDS500. I due **dataset** svolgono ruoli differenti all'interno del lavoro, essendo impiegati rispettivamente per l'addestramento dei modelli e per la fase di test e valutazione.

2.1 BIPED – Barcelona Images for Perceptual Edge Detection

Il **dataset** BIPED (Barcelona Images for Perceptual **Edge Detection**) è stato utilizzato come **dataset** di addestramento per entrambi i modelli di **edge detection** basati su **deep learning** implementati nel progetto, ovvero **TEED** e DexiNED.

BIPED è stato progettato specificamente per il compito di perceptual **edge detection**, fornendo annotazioni di alta qualità che riflettono in modo accurato la percezione umana dei contorni significativi presenti nelle immagini.

Caratteristiche principali:

- * Il **dataset** è composto da 250 immagini ad alta risoluzione (1280 × 720 pixel), raffiguranti principalmente scene urbane esterne, come strade, edifici, marciapiedi ed elementi naturali.

- * Le immagini sono accompagnate da mappe di bordo precise e pulite, realizzate da annotatori esperti e successivamente raffinate per ridurre rumore e ambiguità.

- * Il **dataset** è suddiviso in:

- * 200 immagini per l'addestramento

- * 50 immagini per il test

- * Le annotazioni si concentrano su bordi semanticamente e percettivamente rilevanti, evitando dettagli superflui legati a texture o rumore.

Grazie all'elevata qualità delle annotazioni e alla risoluzione delle immagini, BIPED risulta particolarmente adatto all'addestramento di reti neurali profonde. Per questo motivo è stato scelto come **dataset** principale per l'allenamento dei modelli **TEED** e DexiNED utilizzati nel progetto.

Link di download

Kaggle:

<https://www.kaggle.com/datasets/xavysp/biped>

2.2 BSDS500 – Berkeley Segmentation Dataset

Il **dataset** BSDS500 (Berkeley Segmentation Dataset) è stato utilizzato esclusivamente per scopi di test e valutazione delle prestazioni degli algoritmi di **edge detection**.

Si tratta di uno dei **dataset** di benchmark più diffusi nella letteratura scientifica per la valutazione di algoritmi di segmentazione e rilevamento dei bordi.

Caratteristiche principali:

- * Il **dataset** contiene 500 immagini naturali, che includono una grande varietà di scene, come paesaggi, animali, oggetti e ambienti interni.

- * Le immagini sono suddivise in:

- * 200 immagini di training

- * 100 immagini di validation

- * 200 immagini di test

* Ogni immagine è annotata da più annotatori umani (generalmente tra 3 e 6), producendo diverse mappe di bordo per la stessa immagine.

* Questa molteplicità di annotazioni permette di rappresentare la variabilità soggettiva della percezione umana dei contorni.

Nel contesto di questo progetto, BSDS500 non è stato utilizzato per l'addestramento dei modelli, ma come **dataset** di riferimento per la valutazione della capacità di **edge detection**.

Link di download

Kaggle:

<https://www.kaggle.com/datasets/balraj98/bsds500>

3. Algoritmi Utilizzati

3.1 Algoritmo di Canny

L'**algoritmo di Canny** è una delle tecniche classiche più diffuse ed efficaci per il rilevamento dei bordi nelle immagini digitali. Proposto da John F. **Canny** nel 1986, è stato progettato per individuare contorni ben definiti massimizzando il rapporto segnale-rumore, garantendo una buona localizzazione dei bordi e riducendo al minimo le risposte multiple a uno stesso contorno.

Nel presente progetto, l'**algoritmo di Canny** è stato implementato manualmente in Python, senza l'utilizzo di funzioni di alto livello predefinite, al fine di comprenderne in modo approfondito il funzionamento e il comportamento dei singoli passaggi.

Principali fasi dell'**algoritmo**

L'**algoritmo di Canny** si articola in una sequenza di passaggi ben definiti:

1. Riduzione del rumore

L'immagine di input viene filtrata mediante un filtro gaussiano per attenuare il rumore e le variazioni di intensità non significative che potrebbero generare falsi bordi.

2. Calcolo del gradiente

Vengono calcolate l'intensità e la direzione del gradiente dell'immagine, generalmente attraverso operatori derivativi, al fine di individuare le zone a maggiore variazione di intensità.

3. Soppressione dei non-massimi (Non-Maximum Suppression)

Questa fase permette di assottigliare i bordi, mantenendo solo i massimi locali del gradiente lungo la direzione del bordo ed eliminando le risposte non significative.

4. Doppia sogliatura (Double Thresholding)

I pixel vengono classificati in bordi forti, bordi deboli e non-bordi, sulla base di due soglie (alta e bassa) applicate all'intensità del gradiente.

5. Hysteresis Thresholding

I bordi deboli vengono mantenuti solo se connessi a bordi forti, consentendo di ottenere contorni continui e riducendo ulteriormente il rumore.

Approfondimento nel progetto

Nel contesto di questo progetto, l'**algoritmo** di **Canny** è analizzato in modo dettagliato all'interno di un Python Notebook dedicato, in cui ogni fase viene descritta e implementata passo per passo, con il supporto di visualizzazioni intermedie e commenti esplicativi. Questo approccio consente una comprensione approfondita del funzionamento dell'**algoritmo** e delle scelte progettuali adottate.

3.2 TEED — Tiny Efficient Edge Detector

Il modello Tiny and Efficient Edge Detector (**TEED**) è un approccio di **edge detection** basato su **deep learning** presentato nel paper “Tiny and Efficient Model for the **Edge Detection** Generalization” accettato come contributo al workshop dell'ICCV 2023. **TEED** è stato progettato per affrontare i tradizionali compromessi tra accuratezza, efficienza e generalizzazione nei modelli di **edge detection** basati su reti neurali convoluzionali.

Il contributo principale del modello consiste nel proporre un'architettura molto leggera, con circa 58.000 parametri, meno dello 0,2% rispetto ad altri modelli **deep learning** di stato dell'arte. Questa compattezza permette di ottenere un processo di addestramento e inferenza estremamente veloce, con tempi di training inferiori a 30 minuti su **dataset** come BIPED e con ogni epoca di apprendimento inferiore ai 5 minuti, pur mantenendo una qualità delle mappe di bordo comparabile a modelli più complessi.

Funzionamento del modello **TEED**

Il funzionamento di **TEED** si basa sull'estrazione progressiva di feature di basso e medio livello utili all'individuazione dei bordi, attraverso una rete convoluzionale compatta progettata specificamente per questo compito.

Il modello apprende a distinguere i contorni percettivamente rilevanti dalle strutture di rumore sfruttando convoluzioni leggere e una combinazione efficace delle informazioni spaziali.

Durante la fase di forward pass, l'immagine di input viene trasformata in una rappresentazione interna che enfatizza le discontinuità di intensità e le strutture

geometriche principali.

Queste informazioni vengono poi aggregate e proiettate in una singola mappa di probabilità dei bordi, in cui ogni pixel rappresenta la probabilità di appartenere a un contorno. A differenza degli algoritmi classici, **TEED** non utilizza soglie esplicite o operatori derivativi, ma apprende direttamente dai dati quali pattern visivi corrispondono a bordi significativi, rendendo il modello più robusto a variazioni di illuminazione, texture e rumore. L'elevata capacità di generalizzazione del modello è ottenuta grazie alla semplicità dell'architettura e all'addestramento su annotazioni di alta qualità, che permettono alla rete di focalizzarsi sugli elementi strutturali essenziali dell'immagine.

Principi e struttura

TEED è progettato attorno a tre principi fondamentali:

- * Semplicità: un'architettura CNN progettata per minimizzare la complessità senza compromettere la capacità di rilevamento dei contorni;
- * Efficienza: riduzione drastica del numero di parametri e delle operazioni computazionali, rendendo il modello adatto anche a dispositivi con risorse limitate;
- * Generalizzazione: capacità di funzionare bene su immagini non viste durante l'addestramento, grazie alla combinazione di feature multi-livello e moduli di fusione adeguati.

L'architettura incorpora connessioni skip e moduli di fusione che aiutano a combinare le informazioni estratte a diverse profondità della rete, producendo mappe di bordo nitide e ben definite. Queste caratteristiche permettono a **TEED** di convergere rapidamente durante l'allenamento e generare predizioni di buona qualità con una struttura molto più compatta rispetto a modelli convenzionali.

Allenamento e utilizzo

Nel paper gli autori descrivono l'addestramento del modello utilizzando il **dataset** ****BIPED****, scelto per la sua annotazione accurata dei bordi e per la sua capacità di fornire una base robusta per l'apprendimento supervisionato. Il modello **TEED** si adatta bene ai dati grazie alla sua capacità di convergere rapidamente e di catturare le principali caratteristiche dei contorni percettivamente significativi. Una volta addestrato, **TEED** può essere utilizzato per generare edge map su nuove immagini, producendo risultati visivi di buona qualità con costi computazionali significativamente inferiori rispetto a modelli più pesanti.

Riferimento originale

Il paper completo è disponibile su arXiv e nella raccolta dei workshop dell'ICCV 2023:

Xavier Soria, Yachuan Li, Mohammad Rouhani, Angel D. Sappa.

Tiny and Efficient Model for the **Edge Detection** Generalization, *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2023.

3.3 DexiNed – Dense Extreme Inception Network for Edge Detection

Il modello **DexiNed** (Dense Extreme Inception Network for **Edge Detection**) è un **algoritmo** di **edge detection** basato su una rete neurale convoluzionale profonda, progettato per generare mappe di bordi ad alta qualità con una struttura multi-scala e una forte capacità di generalizzazione. **DexiNed** è stato introdotto dagli autori ****Xavier Soria, Angel Sappa, Patricio Humanante e Arash Akbarinia****, e la sua versione preliminare è stata resa pubblica sotto forma di articolo pre-print su arXiv nel 2021 (Dense Extreme Inception Network for **Edge Detection**).

A differenza di molti modelli di **edge detection** basati su reti profonde che si fondano su architetture di classificazione pre-allenate, **DexiNed** è progettato per essere allenato da zero, senza richiedere pesi pre-addestrati su compiti correlati. Questo consente di adattare l'architettura specificamente al problema della rilevazione dei contorni, focalizzandosi sulla qualità delle mappe di bordo piuttosto che sulla generalità delle rappresentazioni apprese.

Funzionamento dell'**algoritmo**

L'idea centrale del modello è che i bordi non siano un concetto univoco: alcuni sono ben definiti a livello locale, altri emergono solo considerando un contesto più ampio.

L'architettura di **DexiNed** è costruita come una rete convoluzionale profonda con connessioni dense, che permettono di fondere caratteristiche estratte a vari livelli della rete. Ogni stadio produce una edge map intermedia, e queste mappe vengono poi supervisionate direttamente durante l'addestramento (deep supervision). Questo approccio consente al modello di apprendere bordi sia sottili che strutturalmente complessi, riducendo il problema della perdita di dettagli tipico delle reti molto profonde.

Un elemento chiave di **DexiNed** è l'uso di moduli Inception estremi, che combinano filtri convoluzionali di dimensioni diverse all'interno dello stesso blocco. In questo modo il modello riesce a catturare:

- * dettagli fini (edge locali),
- * strutture globali (contorni di oggetti),
- * e variazioni di scala all'interno della stessa immagine.

Durante la fase di inferenza, le diverse edge map prodotte ai vari livelli vengono fuse per ottenere una mappa finale dei bordi più stabile, continua e robusta al rumore. Questo rende **DexiNed** particolarmente efficace su scene complesse, con illuminazione variabile o texture ricche.

Nel contesto del progetto, **DexiNed** viene utilizzato come metodo di **edge detection** basato su modelli pre-addestrati, allenati principalmente sul **dataset** BIPED, mentre il **dataset** BSDS500 viene impiegato per valutare le prestazioni in fase di test e confronto con altre tecniche, come **Canny** e **TEED**.

La rete è composta da due sottoreti principali:

* Una sottorete principale (Dexi) che riceve l'immagine RGB in input e genera una serie di feature map a diverse risoluzioni.

* Una sottorete di upsampling che utilizza queste feature map per generare mappe intermedie dei bordi a ciascun livello, le quali vengono poi fuse per produrre la mappa finale.

Questo approccio multi-livello e multi-scala consente a **DexiNed** di generare mappe di edge finale con bordi molto sottili e continui, spesso superiori rispetto a molte altre architetture basate su reti neurali convenzionali.

Addestramento e generalizzazione

Nel paper originale, **DexiNed** è stato addestrato su **dataset** specificamente curati per la rilevazione dei bordi — in particolare BIPED (Barcelona Images for Perceptual **Edge Detection**) — che fornisce annotazioni di contorni di alta qualità. Questo addestramento da zero permette al modello di apprendere direttamente dalla struttura dei bordi piuttosto che da caratteristiche generali estratte da modelli pre-allenati su compiti più generali come la classificazione di immagini.

Una caratteristica importante di **DexiNed** è la sua capacità di ****generalizzare a dataset diversi da quello di addestramento****, come BSDS500 o MDBD, senza necessità di ulteriori fasi di fine-tuning. Questo comportamento evidenzia la robustezza del modello nel catturare caratteristiche di contorno utili in contesti visivi diversi.

Riferimento al paper ufficiale

Xavier Soria, Angel Sappa, Patricio Humanante, Arash Akbarinia

Dense Extreme Inception Network for **Edge Detection**

arXiv pre-print, Dicembre 2021. ([arXiv][1])

Conclusioni

In questo progetto è stato studiato il problema dell' **edge detection**, analizzando e applicando diverse tecniche appartenenti sia alla visione artificiale classica sia al **deep learning**. In particolare, sono stati presi in esame l'**algoritmo di Canny**, implementato manualmente in Python per comprenderne a fondo il funzionamento, e due modelli basati su reti neurali profonde, **TEED** e **DexiNed**, utilizzati tramite modelli pre-addestrati.

Il lavoro si è concentrato sull'integrazione di questi algoritmi all'interno di un'unica applicazione interattiva, che permette di sperimentarne il comportamento su immagini singole e su **dataset** strutturati. L'obiettivo principale è stato quello di osservare qualitativamente le differenze tra gli approcci, mettendo in evidenza come i metodi classici e quelli basati su **deep learning** affrontino il problema dell'estrazione dei bordi in modo differente. Non è stata effettuata una valutazione quantitativa tramite metriche o score numerici, ma l'attenzione è stata rivolta all'analisi visiva dei risultati e alla comprensione delle caratteristiche e dei punti di forza di ciascun metodo.

4. Architettura del software e funzionamento dell'interfaccia grafica

Per facilitare la sperimentazione e il confronto tra i diversi algoritmi di **edge detection** è stata sviluppata un'interfaccia grafica interattiva in Python utilizzando la libreria Tkinter. L'interfaccia rappresenta il punto di accesso principale al software e consente all'utente di interagire con gli algoritmi in modo semplice e intuitivo.

Struttura dell'interfaccia

L'interfaccia è suddivisa in tre componenti principali:

Pannello di controllo

Il pannello laterale consente di caricare immagini, selezionare l'**algoritmo** da utilizzare e impostare i parametri dell'**algoritmo Canny**.

Area di visualizzazione

La parte centrale dell'applicazione mostra l'immagine di input e il risultato dell'**edge detection** utilizzando Matplotlib integrato in Tkinter.

Sistema di log

Il programma include una sezione di log che permette di monitorare lo stato dell'esecuzione degli algoritmi.

Funzionamento del codice della UI

Il codice dell'interfaccia è organizzato attorno alla classe principale EdgeDetApp che gestisce la creazione dell'interfaccia, il caricamento delle immagini e l'esecuzione degli algoritmi.

Le principali funzioni del codice sono:

load_image() – carica un'immagine dal file system.

on_algorithm_change() – aggiorna dinamicamente i controlli dell'interfaccia.

run_algorithm() – seleziona quale **algoritmo** eseguire.

run_canny() – esegue l'**algoritmo Canny**.

run_teed() e run_dexined() – eseguono i modelli **deep learning**.

show_teed_image() e show_dexined_image() – visualizzano i risultati.

Per evitare che l'interfaccia grafica si blocchi durante l'esecuzione dei modelli **deep learning** vengono utilizzati thread separati.

Inoltre, alla chiusura dell'applicazione viene eseguita una procedura di pulizia delle cartelle temporanee dei modelli per evitare l'accumulo di file inutili.